Assignment A1 : Tokenization and Statistical Language Modelling

Individual assignment • 20% individual-report component Released: 19 Nov 2026 Due: 26 Nov 2026, 23:59

In class we saw that **tokenization** is the first step in text processing, and that a **language model** assigns a probability to text — the better it models language, the more fluent the text it generates. In this assignment you will build **your own tokenizer and a simple statistical (*n*-gram) language model**, use it to generate text, and compare it against **GPT-2**. The question to answer: *how fluent is your statistical model, and why do statistical language models fall short of neural ones like GPT-2?*

Data. We use the comment text from the Kaggle competition *Jigsaw Unintended Bias in Toxicity Classification* (https://www.kaggle.com/c/jigsaw-unintended-bias-in-toxicity-classification). Load it with `load_lm()` from the `starter/` bundle (join the competition first and set up Kaggle credentials). *Content note: the comments can be offensive.* It returns a training split and a held-out split of comments. Everything runs on a laptop CPU in a few minutes — GPT-2 small only needs a forward pass, no training.

1. Tasks

1. **Build your model.** Train a BPE tokenizer on the training text, then build a smoothed bigram or trigram language model over those tokens. Use it to **generate a few sample sentences**.
2. **Bring in GPT-2.** Load GPT-2 small (`gpt2`) and generate sample sentences the same way, so you can compare its fluency to your model's.
3. **Measure and explain.** Compute **perplexity** on the held-out split for both your *n*-gram model and GPT-2, and explain — using your samples and the two numbers — why the statistical model is less fluent.

What to submit & how it's graded. Submit `a1_perplexity.json` with your two numbers (`ngram_ppl`, `gpt2_ppl`) and a short `samples.txt` with your generated text. The two perplexities are checked against a hidden reference within tolerance; the samples and your explanation are read for the findings. *Metric: perplexity (lower is better).*

2. Reflection (required, max 200 words)

Submit `reflection.md` (or `.pdf`). It must answer these three fixed questions and include the AI-use declaration. The reflection is graded for honesty and insight, not length.

1. **What did you know before?** — your starting point on this topic.
2. **What was hardest, and how did you get unstuck?**
3. **What does your result say, and what would you try next?**

AI-use declaration. State which parts (if any) were generated with AI tools, and *how you verified them.* Using AI is allowed; undisclosed or unverified use is not.

3. Submission

Upload a single `zip` to Brightspace named `A1_<studentnumber>.zip`, containing the required output file(s) listed above, your `code/` folder, and `reflection.md`. Run the format checker in `starter/` on your zip first — it validates file names, schemas, and value ranges locally before you

submit. Auto-grading scores your output file against the hidden reference; the format checker passing is a precondition for grading.